Database Design Guide for Data Engineers

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:05 AM PDT

Database Design Guide for Data Engineers

Overview

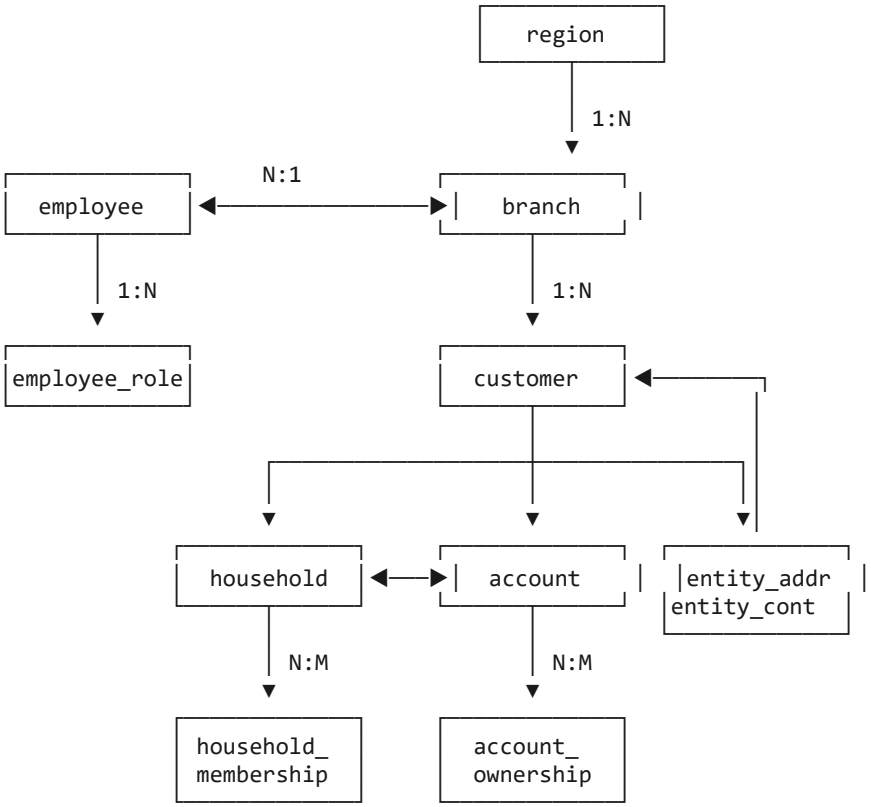
This document provides a comprehensive understanding of the ClientIQ database schema, relationships, and data model. It is intended for Data Engineers who need to understand the data architecture for analytics, reporting, ETL processes, and integration work.

Schema Summary

The ClientIQ database consists of **39 tables** organized into functional domains:

Domain	Tables	Description
Core Entities	8	Customers, accounts, employees, branches
Relationships	7	Junction tables for many-to-many relationships
Transactions	3	Financial transactions and categories
Notes	4	Customer and account notes with versioning
RBAC	10	Role-based access control
Online Banking	3	OLB users and login events
Cards	2	Debit cards and limits
Reference	2	SIC codes and regions

Entity Relationship Diagram



Core Tables

Region Table

Geographic regions for branch organization.

```
CREATE TABLE region (
  region_id BIGINT PRIMARY KEY,         -- Auto-increment
  region_name VARCHAR(100) NOT NULL,    -- e.g., "Northwest"
  region_code VARCHAR(10) UNIQUE,       -- e.g., "NW"
  created_at DATETIME2,
  updated_at DATETIME2
);
```

Sample Data:

region_id	region_name	region_code
1	Northwest	NW
2	Southwest	SW
3	Central	CT

Branch Table

Bank branch locations.

```
CREATE TABLE branch (
  branch_id BIGINT PRIMARY KEY,
  branch_code VARCHAR(10) UNIQUE,      -- Internal code
  branch_name VARCHAR(100) NOT NULL,
  branch_type VARCHAR(20),             -- main, satellite, virtual
  address_id BIGINT FK,                -- Physical location
  region_id BIGINT FK,                 -- Region grouping
  is_active BIT,
  opened_date DATE,
  created_at DATETIME2,
  updated_at DATETIME2
);
```

Employee Table

Bank staff with SAML SSO integration.

```
CREATE TABLE employee (
  employee_id BIGINT PRIMARY KEY,
  employee_number VARCHAR(20) UNIQUE,  -- Badge number
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  title VARCHAR(50),                   -- Mr., Ms., Dr.
  position VARCHAR(100),               -- Job title
  officer_code VARCHAR(20) UNIQUE,     -- For officer assignments
  department VARCHAR(50),
  is_active BIT,
  hire_date DATE,

  -- SAML SSO Fields
  sso_subject VARCHAR(255) UNIQUE,     -- SAML NameID
  email VARCHAR(255),
  phone VARCHAR(20),
  last_seen_saml_role VARCHAR(255),    -- Last role from IdP
  last_login_at DATETIME2,
  deleted_at DATETIME2,                -- Soft delete
  modified_by BIGINT FK,               -- Audit trail

  created_at DATETIME2,
  updated_at DATETIME2
);
```

Key Points:

- `employee_id` is the primary identifier, sourced from SAML
- `officer_code` links to customer assignments
- SSO fields track authentication state

Customer Table (Polymorphic)

Supports individual, business, and trust customers.

```sql
CREATE TABLE customer (
  customer_id BIGINT PRIMARY KEY,

  -- Individual Fields (used when customer_type IN ('individual', 'premium', 'regular'))
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  middle_name VARCHAR(100),
  preferred_name VARCHAR(100),
  title VARCHAR(20),
  suffix VARCHAR(20),
  date_of_birth DATE,
  gender VARCHAR(20),
  marital_status VARCHAR(20),

  -- Business/Trust Fields (used when customer_type IN ('business', 'trust'))
  business_name VARCHAR(200),
  dba_name VARCHAR(200),               -- Doing Business As

  -- Unified Search Field
  full_name VARCHAR(200),              -- Computed/stored for search

  -- Identification
  tax_identifier VARCHAR(20),          -- SSN/EIN (masked in UI)
  government_id VARCHAR(50),           -- Drivers license, etc.
  government_id_type VARCHAR(20),
  citizenship VARCHAR(50),

  -- Classification
  customer_type VARCHAR(20) NOT NULL,  -- individual, premium, regular, business, trust
  customer_status VARCHAR(20),         -- active, inactive, deceased, closed
  customer_since DATE,

  -- Compliance
  kyc_status VARCHAR(20),              -- verified, pending, expired
  kyc_last_updated DATE,
  risk_rating VARCHAR(20),             -- low, medium, high, critical

  -- Professional (individuals)
  occupation VARCHAR(100),
  employer_name VARCHAR(200),

  -- Business Classification
  naics_code VARCHAR(10),

  -- Branch and Core Banking
  branch_id BIGINT FK,
  jack_henry_cif_number VARCHAR(20),   -- Core banking ID
  silverlake_customer_id VARCHAR(20),  -- Alternate core ID

  -- Internal Codes
  inquiry_code VARCHAR(20),
  inside_code VARCHAR(20),
  sales_associate_code VARCHAR(20),
  class_code VARCHAR(20),

  -- Flags
  is_employee BIT,                     -- Bank employee
  vip_customer BIT,                    -- VIP designation
  is_deceased BIT,

  created_at DATETIME2,
  updated_at DATETIME2
);
```

Customer Type Logic:

```
-- Validation rule (enforced in application):
-- Individual customers: first_name, last_name required; business_name NULL
-- Business/Trust: business_name required; first_name, last_name NULL
```

Household Table

Family or organizational groupings with B2B support.

```
CREATE TABLE household (
  household_id BIGINT PRIMARY KEY,
  household_name VARCHAR(200) NOT NULL,
  household_type VARCHAR(50),          -- family, business, trust
  total_assets DECIMAL(15,2),
  total_liabilities DECIMAL(15,2),
  household_status VARCHAR(20),        -- active, inactive
  risk_rating VARCHAR(20),
  relationship_manager_id BIGINT FK,   -- Primary RM
  established_date DATE,
  tax_filing_status VARCHAR(20),

  -- B2B Relationship Fields
  parent_household_id BIGINT FK,       -- Parent organization (self-ref)
  consolidation_method VARCHAR(20),    -- full, equity, proportionate, none

  created_at DATETIME2,
  updated_at DATETIME2,

  CONSTRAINT chk_no_self_parent CHECK (household_id <> parent_household_id)
);
```

B2B Hierarchy Example:

```
Acme Corporation (parent_household_id: NULL)
├── Acme Manufacturing (parent_household_id: 1)
├── Acme Distribution (parent_household_id: 1)
│   └── Acme Logistics (parent_household_id: 2)
└── Acme Finance (parent_household_id: 1)
```

Account Table

Bank accounts of all types.

```
CREATE TABLE account (
  account_id BIGINT PRIMARY KEY,
  account_number VARCHAR(50) UNIQUE,
  account_type VARCHAR(50) NOT NULL,    -- checking, savings, money_market, cd, loan, credit_card
  account_subtype VARCHAR(50),          -- personal_checking, business_checking, etc.
  account_status VARCHAR(20) NOT NULL, -- active, dormant, closed, frozen
  balance DECIMAL(15,2),
  available_balance DECIMAL(15,2),
  currency VARCHAR(3) DEFAULT 'USD',
  interest_rate DECIMAL(5,4),
  credit_limit DECIMAL(15,2),           -- For credit products
  branch_id BIGINT FK,
  product_code VARCHAR(50),
  opened_date DATE,
  closed_date DATE,
  last_transaction_date DATE,
  maturity_date DATE,                   -- For CDs

  -- Core Banking Integration
  jack_henry_account_id VARCHAR(50),
  silverlake_account_structure VARCHAR(200),

  -- Statement and Maintenance
  account_class VARCHAR(50),
  statement_cycle VARCHAR(20),
  statement_code_desc VARCHAR(200),
  average_balance DECIMAL(15,2),
  last_maintenance_date DATE,

  created_at DATETIME2,
  updated_at DATETIME2
);
```

Relationship Tables

Household Membership

Links customers to households with ownership tracking.

```
CREATE TABLE household_membership (
  membership_id BIGINT PRIMARY KEY,
  household_id BIGINT FK NOT NULL,
  customer_id BIGINT FK NOT NULL,
  relationship_role VARCHAR(50) NOT NULL,  -- head, spouse, child, beneficiary, trustee
  is_primary_member BIT,
  is_head_of_household BIT,
  membership_start_date DATE,
  membership_end_date DATE,
  rollup_accounts BIT DEFAULT 1,
  rollup_percentage DECIMAL(5,2) DEFAULT 100,

  -- B2B Ownership Fields
  ownership_percentage DECIMAL(5,2) NOT NULL DEFAULT 0,  -- 0-100%
  control_type VARCHAR(30) DEFAULT 'none',  -- majority_control, significant_influence, minority, none

  notes NVARCHAR(MAX),
  created_at DATETIME2,
  updated_at DATETIME2
);
```

Control Type Rules:

Ownership %	Control Type
> 50%	majority_control
20-50%	significant_influence
< 20%	minority
0%	none

Account Ownership

Links customers to accounts with permissions.

```
CREATE TABLE account_ownership (
  ownership_id BIGINT PRIMARY KEY,
  account_id BIGINT FK NOT NULL,
  customer_id BIGINT FK NOT NULL,
  ownership_type VARCHAR(50) NOT NULL,    -- primary, joint, authorized_signer, beneficiary
  ownership_percentage DECIMAL(5,2),
  is_primary_owner BIT,
  signing_authority BIT,
  can_view_statements BIT,
  can_make_transactions BIT,
  transaction_limit DECIMAL(15,2),
  relationship_start_date DATE,
  relationship_end_date DATE,

  created_at DATETIME2,
  updated_at DATETIME2
);
```

Entity Address/Contact

Polymorphic junction tables for addresses and contacts.

```sql
-- Addresses
CREATE TABLE entity_address (
  entity_address_id BIGINT PRIMARY KEY,
  entity_type VARCHAR(20) NOT NULL,    -- customer, branch, employee
  entity_id BIGINT NOT NULL,           -- References the entity
  address_id BIGINT FK NOT NULL,       -- References address table
  address_purpose VARCHAR(20),         -- primary, mailing, billing, work
  is_current BIT,
  start_date DATE,
  end_date DATE
);

-- Contacts
CREATE TABLE entity_contact (
  entity_contact_id BIGINT PRIMARY KEY,
  entity_type VARCHAR(20) NOT NULL,    -- customer, branch, employee
  entity_id BIGINT NOT NULL,
  contact_id BIGINT FK NOT NULL,       -- References contact_info table
  contact_purpose VARCHAR(20),         -- primary, work, emergency
  is_current BIT,
  start_date DATE,
  end_date DATE
);
```

Transaction Tables

Financial Transaction

All account transactions.

```
CREATE TABLE financial_transaction (
  transaction_id BIGINT PRIMARY KEY,
  account_id BIGINT FK NOT NULL,
  amount DECIMAL(15,2) NOT NULL,       -- Positive=credit, Negative=debit
  transaction_code VARCHAR(30),
  transaction_type VARCHAR(30),        -- deposit, withdrawal, transfer, payment
  status VARCHAR(20) NOT NULL,         -- pending, posted, reversed
  transaction_date DATETIME2 NOT NULL,
  posting_date DATETIME2 NOT NULL,
  description NVARCHAR(MAX),
  reference_number VARCHAR(64),
  merchant_name VARCHAR(200),
  merchant_category_code VARCHAR(4),   -- MCC
  category_id BIGINT FK,               -- Transaction category

  -- Transfer Tracking
  transfer_group_id UNIQUEIDENTIFIER,  -- Groups related transfers
  counterparty_account_id BIGINT FK,   -- Destination/source account
  related_transaction_id BIGINT FK,    -- Reversed transaction, etc.

  -- Balance Snapshots
  ledger_balance_after DECIMAL(15,2) NOT NULL,
  available_balance_after DECIMAL(15,2) NOT NULL,

  -- Source System
  source_system VARCHAR(32) NOT NULL,  -- jack_henry, silverlake, manual
  source_transaction_id VARCHAR(128),
  raw_payload NVARCHAR(MAX),           -- Original JSON

  created_at DATETIME2,
  updated_at DATETIME2,

  UNIQUE (account_id, source_system, source_transaction_id)
);
```

Index Strategy:

```
-- Primary query patterns
CREATE INDEX idx_txn_account_posting ON financial_transaction(account_id, posting_date, transaction_id);
CREATE INDEX idx_txn_account_status ON financial_transaction(account_id, status, posting_date);
CREATE INDEX idx_txn_transfer_group ON financial_transaction(transfer_group_id);
```

Notes Module

Note and Note Version

Immutable audit trail for customer/account notes.

```sql
-- Note identity (immutable target)
CREATE TABLE note (
  note_id BIGINT PRIMARY KEY,
  customer_id BIGINT FK,               -- Mutually exclusive with account_id
  account_id BIGINT FK,
  target_type VARCHAR(20) NOT NULL,    -- customer, account
  category_id BIGINT FK,
  importance VARCHAR(20) NOT NULL,     -- low, medium, high, urgent
  visibility VARCHAR(20) NOT NULL,     -- public, internal, confidential
  legal_hold BIT,                      -- Prevents deletion
  retention_years BIGINT,              -- NULL = indefinite
  is_pinned BIT,

  created_at DATETIME2,
  updated_at DATETIME2,

  CONSTRAINT chk_note_one_target CHECK (
    (customer_id IS NOT NULL AND account_id IS NULL) OR
    (customer_id IS NULL AND account_id IS NOT NULL)
  )
);

-- Note content (versioned)
CREATE TABLE note_version (
  version_id BIGINT PRIMARY KEY,
  note_id BIGINT FK NOT NULL,
  version_number BIGINT NOT NULL,
  title VARCHAR(200) NOT NULL,
  body NVARCHAR(MAX) NOT NULL,
  author_employee_id BIGINT FK NOT NULL,
  author_employee_name VARCHAR(200),   -- Denormalized
  is_current BIT,                      -- Only one current per note
  is_soft_deleted BIT,
  deleted_at DATETIME2,
  deleted_by_employee_id BIGINT FK,
  encrypted_payload NVARCHAR(MAX),     -- For confidential notes

  created_at DATETIME2,
  modified_at DATETIME2
);
```

Version History Query:

```sql
SELECT nv.version_number, nv.title, nv.author_employee_name, nv.created_at
FROM note_version nv
WHERE nv.note_id = @noteId
ORDER BY nv.version_number DESC;
```

RBAC Tables

Role-Based Access Control

```sql
-- Privilege levels (1-4)
CREATE TABLE privilege_level (
  level BIGINT PRIMARY KEY,
  level_name VARCHAR(50) UNIQUE,       -- Level 1 - Basic, etc.
  description NVARCHAR(MAX)
);

-- Roles
CREATE TABLE role (
  role_id BIGINT PRIMARY KEY,
  role_name VARCHAR(100) UNIQUE,
  privilege_level BIGINT FK NOT NULL,
  description NVARCHAR(MAX),
  is_system_role BIT,                  -- Cannot be deleted
  is_active BIT
);

-- Permissions (ABAC-enabled)
CREATE TABLE permission (
  permission_id BIGINT PRIMARY KEY,
  permission_code VARCHAR(100) UNIQUE, -- customer.view, account.edit
  resource VARCHAR(50) NOT NULL,
  action VARCHAR(50) NOT NULL,
  description NVARCHAR(MAX),
  min_privilege_level BIGINT FK,
  is_attribute_based BIT,              -- Uses ABAC rules
  attribute_config NVARCHAR(MAX),      -- JSON rules
  is_active BIT
);

-- Role-Permission mapping
CREATE TABLE role_permission (
  role_id BIGINT,
  permission_id BIGINT,
  granted_at DATETIME2,
  granted_by BIGINT FK,
  PRIMARY KEY (role_id, permission_id)
);

-- Employee-Role assignment
CREATE TABLE employee_role (
  employee_id BIGINT,
  role_id BIGINT,
  is_primary BIT,
  assigned_by BIGINT FK,
  assigned_date DATETIME2,
  effective_date DATE,
  expiration_date DATE,
  is_active BIT,
  PRIMARY KEY (employee_id, role_id)
);
```

Permission Check Query:

```sql
-- Get all active permissions for an employee
SELECT DISTINCT p.permission_code, p.resource, p.action, p.is_attribute_based, p.attribute_config
FROM employee_role er
JOIN role r ON r.role_id = er.role_id
JOIN role_permission rp ON rp.role_id = r.role_id
JOIN permission p ON p.permission_id = rp.permission_id
WHERE er.employee_id = @employeeId
  AND er.is_active = 1
  AND r.is_active = 1
  AND p.is_active = 1
  AND (er.expiration_date IS NULL OR er.expiration_date > GETDATE());
```

Common Query Patterns

Customer 360 View

```sql
-- Get complete customer summary
SELECT
  c.customer_id,
  c.full_name,
  c.customer_type,
  c.customer_status,
  c.vip_customer,
  c.is_employee,
  b.branch_name,

  -- Aggregated metrics
  (SELECT COUNT(*) FROM account_ownership ao WHERE ao.customer_id = c.customer_id) AS account_count,
  (SELECT SUM(a.balance) FROM account a
   JOIN account_ownership ao ON ao.account_id = a.account_id
   WHERE ao.customer_id = c.customer_id AND a.account_status = 'active') AS total_balance,

  -- Primary contact
  (SELECT TOP 1 ci.contact_value FROM contact_info ci
   JOIN entity_contact ec ON ec.contact_id = ci.contact_id
   WHERE ec.entity_type = 'customer' AND ec.entity_id = c.customer_id
   AND ci.contact_type = 'phone' AND ci.is_primary = 1) AS primary_phone

FROM customer c
LEFT JOIN branch b ON b.branch_id = c.branch_id
WHERE c.customer_id = @customerId;
```

Household Hierarchy

```sql
-- Get household tree using recursive CTE
WITH HouseholdHierarchy AS (
  -- Anchor: top-level households
  SELECT household_id, household_name, parent_household_id, 0 AS level,
         CAST(household_name AS NVARCHAR(1000)) AS path
  FROM household
  WHERE parent_household_id IS NULL

  UNION ALL

  -- Recursive: child households
  SELECT h.household_id, h.household_name, h.parent_household_id, hh.level + 1,
         CAST(hh.path + ' > ' + h.household_name AS NVARCHAR(1000))
  FROM household h
  JOIN HouseholdHierarchy hh ON h.parent_household_id = hh.household_id
)
SELECT * FROM HouseholdHierarchy
ORDER BY path;
```

Transaction Summary

```sql
-- Monthly transaction summary by account
SELECT
  a.account_number,
  a.account_type,
  YEAR(ft.posting_date) AS year,
  MONTH(ft.posting_date) AS month,
  COUNT(*) AS transaction_count,
  SUM(CASE WHEN ft.amount > 0 THEN ft.amount ELSE 0 END) AS total_credits,
  SUM(CASE WHEN ft.amount < 0 THEN ABS(ft.amount) ELSE 0 END) AS total_debits,
  AVG(ft.ledger_balance_after) AS avg_balance
FROM financial_transaction ft
JOIN account a ON a.account_id = ft.account_id
WHERE ft.posting_date >= DATEADD(MONTH, -12, GETDATE())
GROUP BY a.account_number, a.account_type, YEAR(ft.posting_date), MONTH(ft.posting_date)
ORDER BY a.account_number, year, month;
```

Data Types Reference

Drizzle Type	PostgreSQL	SQL Server	Notes
`bigserial`	`BIGSERIAL`	`BIGINT IDENTITY(1,1)`	Auto-increment
`bigint`	`BIGINT`	`BIGINT`	64-bit integer
`text`	`TEXT`	`NVARCHAR(MAX)`	Unicode text
`varchar(n)`	`VARCHAR(n)`	`VARCHAR(n)`	ASCII string
`boolean`	`BOOLEAN`	`BIT`	True/false
`timestamp`	`TIMESTAMP`	`DATETIME2`	Date and time
`date`	`DATE`	`DATE`	Date only
`decimal(p,s)`	`DECIMAL(p,s)`	`DECIMAL(p,s)`	Exact numeric
`jsonb`	`JSONB`	`NVARCHAR(MAX)`	JSON data
`uuid`	`UUID`	`UNIQUEIDENTIFIER`	GUID

Next Steps

1. DBA Setup Guide
2. Environment Variables